

Entrega Guía 1: Gestión de tareas

April 30, 2026

Link al repositorio con el código completo: https://github.com/Emi-P/sistemas_de_tiempo_real

1 Ejercicio 1

1.1 Implementación

Las tareas implementadas son todas de esta pinta, con la diferencia del delay utilizado para setear la variable Delay_ms y los LEDs utilizados son uno distinto para cada tarea.

```
void vTareaParpadeo200(void *pvParameters){
const TickType_t Delay_ms = pdMS_TO_TICKS( 200 );
GPIO_TypeDef* port = LD4_GPIO_Port;
uint16_t pin = LD4_Pin;
while (1){
    HAL_GPIO_TogglePin(port, pin);
    HAL_Delay( Delay_ms );
}
}
```

Y se crean las tareas de esta manera

```
xTaskCreate(
    vTareaParpadeo200,
    "Blink200",
    configMINIMAL_STACK_SIZE,
    NULL,
    tskIDLE_PRIORITY,
    NULL
);
... // 400ms, 600ms

xTaskCreate(
    vTareaParpadeo800,
    "Blink200",
    configMINIMAL_STACK_SIZE,
    NULL,
    tskIDLE_PRIORITY,
    NULL
);
```

1.2 Conclusiones

Todas las tareas se ejecutan. Esto es obvio pues cada tarea tiene asignado un LED distinto y estos están funcionando a periodos diferentes.

Pareciera que hay concurrencia pero dado que el procesador es de un solo núcleo, sabemos que es una ilusión provocada por el scheduling de FREERTOS. Mas claramente comprobamos en clase usando el osciloscopio, hay una medible diferencia entre el inicio de una tarea a pesar de que esto es aparentemente en el mismo instante.

Los tiempos de conmutación no serán precisos. Pues dado que las esperas son no-bloqueantes. El scheduler tendrá que intervenir para repartir la cpu y en dichos cambios de contexto se introduce jitter.

2 Ejercicio 2

2.1 Implementación

La única tarea implementada es

```
void vTareaParpadeo(void *pvParameters){
    // Recibir parametros
    struct ParpadeoParameters parameters = \
        *(struct ParpadeoParameters *) pvParameters;

    const TickType_t xDelayms = pdMS_TO_TICKS( parameters.ms );
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        vTaskDelay( xDelayms );
    }
}
```

Y se inician en main, las tareas:

```
// Crear las configuraciones para cada tarea
static struct ParpadeoParameters task200Parameters = {
    .Pin = LD4_Pin,
    .Port = LD4_GPIO_Port,
    .ms = 200
};
// ... 400ms, 600ms, y LEDs 5 y 3
static struct ParpadeoParameters task800Parameters = {
    .Pin = LD6_Pin,
    .Port = LD6_GPIO_Port,
    .ms = 800
};

xTaskCreate(
    vTareaParpadeo,
    "Blink200",
    configMINIMAL_STACK_SIZE,
    &task200Parameters,
    tskIDLE_PRIORITY,
    NULL
);
// ... 400ms, 600ms
xTaskCreate(
    vTareaParpadeo,
    "Blink800",
    configMINIMAL_STACK_SIZE,
    &task800Parameters,
    tskIDLE_PRIORITY,
    NULL
);
```

2.2 Conclusiones

En el Desafío 2 las tareas utilizan `vTaskDelay`, lo que introduce estados de bloqueo (Blocked). A diferencia del Desafío 1, donde las tareas permanecían activas usando busy-wait, ahora ceden el CPU voluntariamente. Esto permite al scheduler gestionar mejor la ejecución, reduce la competencia por CPU y disminuye el jitter. Como resultado, los períodos de conmutación son más estables, aunque siguen limitados por la resolución del tick del sistema.

3 Ejercicio 3

3.1 Implementación

```
void vHandlePush(void *pvParameters) {
    while(1){
        if ( HAL_GPIO_ReadPin(B1_GPIO_Port , B1_Pin) == GPIO_PIN_SET ){
            HAL_GPIO_WritePin(LD6_GPIO_Port , LD6_Pin , GPIO_PIN_SET);
        }
        else if ( HAL_GPIO_ReadPin(B1_GPIO_Port , B1_Pin) ==
GPIO_PIN_RESET ){
            HAL_GPIO_WritePin(LD6_GPIO_Port , LD6_Pin , GPIO_PIN_RESET);
        }
        vTaskDelay(pdMS_TO_TICKS(10)); // Bloquearse 10ms
    }
}

void vTareaParpadeo(void *pvParameters){
    // Recibir parametros
    struct ParpadeoParameters parameters = *(struct ParpadeoParameters *)
pvParameters;
    while (1){
        HAL_GPIO_TogglePin(parameters.Port , parameters.Pin);
        HAL_Delay( 500 );
    }
}
```

`vHandlePush` hará polling cada 10ms para verificar el estado del botón y aplicar el estado correspondiente al LED. La otra tarea se reutilizo de ejercicios pero ahora es no-bloqueante por usar `HAL_Delay` como servicio de espera.

Se crean las tareas de esta manera

```
static struct ParpadeoParameters LedBloqueante500ms = {
    .Pin = LD4_Pin,
    .Port = LD4_GPIO_Port,
    .ms = 500
};

xTaskCreate(
    vTareaParpadeo,
    "LedBloqueante500ms",
    configMINIMAL_STACK_SIZE,
    &LedBloqueante500ms,
    2,
    NULL
);
xTaskCreate(
    vHandlePush,
    "PollingBoton",
    configMINIMAL_STACK_SIZE,
    NULL,
    3,
    NULL
);
```

3.2 Conclusiones

Al usar tiempos de `vTaskDelay` mas altos, el botón responde muy tarde al pulsar y soltar. Y es posible hallar ventanas donde se puede presionar y soltar sin respuesta de ningún tipo. Un `vTaskDelay` de 10ms satisface la consigna, pues asegura que la tarea no estaría más de 10ms bloqueada y sabemos que una vez se ponga `READY`, el scheduler decide ejecutarla. Esto no se asegura con total precision, pues se introduce un jitter por el cambio de contexto, dado que la otra tarea siempre está ejecutando el resto del tiempo.

4 Ejercicio 4

4.1 Implementación

```
void vTareaParpadeoA(void *pvParameters){
    struct ParpadeoParameters parameters = *(struct ParpadeoParameters
*) pvParameters;
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        HAL_Delay( 100 ); // Trabajo simulado
        vTaskDelay(pdMS_TO_TICKS(parameters.ms)); // Handlear cada 10ms
    }
}

void vTareaParpadeoB(void *pvParameters){
    struct ParpadeoParameters parameters = *(struct ParpadeoParameters
*) pvParameters;
    TickType_t xLastWakeTime;
    xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(parameters.ms);
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        HAL_Delay( 100 ); // Trabajo simulado
        vTaskDelayUntil(&xLastWakeTime, xFrequency);
    }
}
```

Notar que la TareaB utiliza vTaskDelayUntil en vez de vTaskDelay como en TareaA.

Las tareas se crean con la misma prioridad, y usando LED's diferentes. Notar que se setea el campo ms de forma diferente

```
static struct ParpadeoParameters paramsTareaA = {
    .Pin = LD4_Pin,
    .Port = LD4_GPIO_Port,
    .ms = 400
};
static struct ParpadeoParameters paramsTareaB = {
    .Pin = LD5_Pin,
    .Port = LD5_GPIO_Port,
    .ms = 500
};

xTaskCreate(
    vTareaParpadeoA,
    "LedBloqueante-Delay500ms",
    configMINIMAL_STACK_SIZE,
    &paramsTareaA,
    0,
    NULL
);
xTaskCreate(
    vTareaParpadeoB,
    "LedBloqueante-DelayUntil500ms",
    configMINIMAL_STACK_SIZE,
    &paramsTareaB,
    0,
    NULL
);
```

4.2 Conclusiones

El tiempo utilizado en la TareaA es de 400ms, pues queremos mantener una periodicidad de 500ms sabiendo que se realiza trabajo de 100ms en el medio. En el caso de TareaB, vTaskDelayUntil es capaz de compensar el retardo agregado por el trabajo de 100ms pues considera el tiempo absoluto desde que inicio la tarea.

Si ambos argumentos son iguales de 500ms, se observa un desfase muy exagerado en la conmutación de los LED's. Dado que la TareaA termina el trabajo de 100ms y es entonces que comienza a esperar 500ms para desbloquearse. En cambio la TareaB considera la diferencia.

5 Ejercicio 5

5.1 Implementación

```
void vTareaParpadeoA(void *pvParameters){
    struct ParpadeoParameters parameters = *(struct ParpadeoParameters
*) pvParameters;

    UBaseType_t prioOriginal = uxTaskPriorityGet(NULL);
    TickType_t tiempoFin = 0;
    BaseType_t enBoost = pdFALSE;

    while (1){
        if (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_SET){
            vTaskPrioritySet(NULL, prioOriginal + 1);
            tiempoFin = xTaskGetTickCount() + pdMS_TO_TICKS(3000);
            enBoost = pdTRUE;
        }

        if (enBoost == pdTRUE && xTaskGetTickCount() >= tiempoFin){
            vTaskPrioritySet(NULL, prioOriginal);
            enBoost = pdFALSE;
        }

        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);

        HAL_Delay(parameters.ms);
    }
}

void vTareaParpadeoB(void *pvParameters){
    struct ParpadeoParameters parameters = *(struct ParpadeoParameters
*) pvParameters;
    TickType_t xLastWakeTime;
    xLastWakeTime = xTaskGetTickCount();
    const TickType_t xFrequency = pdMS_TO_TICKS(parameters.ms);
    while (1){
        HAL_GPIO_TogglePin(parameters.Port, parameters.Pin);
        HAL_Delay(xFrequency);
    }
}
```

La lógica de la TareaB es sencilla. Solamente se encarga de hacer un toggleo 'periodico' del LED.

La TareaA agrega algo de lógica para mantener el parpadeo aunque se esté atravesando los 3 segundos de boosteo de prioridad. Al detectar que se presiona el botón, setea en pdTRUE la variable enBoost que indica que se encuentra durante un boosteo. Además tiempoFin indica el tiempo en que se cumplen 3 segundos de boosteo.

Las esperas en ambas tareas son, no-bloqueantes para observar el starving al tener la TareaA en una prioridad mayor.

5.2 Conclusiones

Al restablecerse la prioridad de la TareaA, la conmutación del LED para la TareaB vuelve a ser normal

El conteo de los 3 segundos es mediante `xTaskGetTickCount()`, de forma que se calcula el tiempo en el que pasaran los 3 segundos. Podría haberse utilizado una espera de 3 segundos usando `HAL_Delay()`, pero con esa implementación no se mantendría el parpadeo de la TareaA. No se puede utilizar `vTaskDelay` ni `vTaskDelayUntil`, pues bloquearían el proceso de la TareaA resultando que la TareaB pueda reanudar en esas ventanas de tiempo a pesar de ser de menor prioridad.

6 Ejercicio 6

6.1 Implementación

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if (GPIO_Pin == GPIO_PIN_0){
        xSemaphoreGiveFromISR(xButtonSemaphore, &
xHigherPriorityTaskWoken);

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}
void vTareaBoton(void *pvParameters){
    while (1){

        xSemaphoreTake(xButtonSemaphore, portMAX_DELAY);

        HAL_GPIO_TogglePin(LD3_GPIO_Port, LD3_Pin);
        vTaskDelay(pdMS_TO_TICKS(50)); // Debounce

    }
}
```

El código del callback es sencillo, habilita la posibilidad de tomar el semáforo cada vez que se levanta la interrupción si fue por haberse presionado el botón en `GPIO_PIN_0`.

`vTareaBoton` está casi siempre bloqueada, pues cada vez que ejecuta `xSemaphoreTake`, debe esperar a que el callback le de la posibilidad de tomarlo nuevamente.

Agrega una pequeña espera para reducir el bounce del botón.

6.2 Conclusiones

Se propone hacer que el LED a encender rote entre los 4 LEDs cada vez que se ejecuta el bucle principal de la tarea.

De esta forma si la tarea estuviera haciendo polling en vez de actuar unicamente por ocurrir una interrupción, sería impredecible el próximo LED que se enciende. Dado que se usa este sistema que bloquea la tarea hasta que se toca el botón sera predecible saber cual es el próximo LED que se enciende al presionar el botón.

Se implementa de esta forma:

```
void vTareaBoton(void *pvParameters) {
    uint16_t leds [] = {LD3_Pin, LD4_Pin, LD5_Pin, LD6_Pin};
    int i = 0;

    while (1) {
        xSemaphoreTake(xButtonSemaphore, portMAX_DELAY);
        HAL_GPIO_WritePin(GPIOD, leds[i], GPIO_PIN_RESET);
        i = (i + 1) % 4;
        HAL_GPIO_WritePin(GPIOD, leds[i], GPIO_PIN_SET);
        vTaskDelay(pdMS_TO_TICKS(50)); // Debounce
    }
}
```

7 Ejercicio 7

7.1 Implementación

Las tareas implmeentadas son tres:

```
void vTareaLED1(void *pvParameters){
    while (1){
        xSemaphoreTake( xSem1, portMAX_DELAY );
        HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_SET);
        xSemaphoreGive( xSem2 );
    }
}
void vTareaLED2(void *pvParameters){
    while (1){
        xSemaphoreTake( xSem2, portMAX_DELAY );
        HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LD4_GPIO_Port, LD4_Pin, GPIO_PIN_SET);
        xSemaphoreGive( xSem3 );
    }
}
void vTareaLED3(void *pvParameters){
    while (1){
        xSemaphoreTake( xSem3, portMAX_DELAY );
        HAL_GPIO_WritePin(LD4_GPIO_Port, LD4_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_SET);
        xSemaphoreGive( xSem1 );
    }
}
```

Cada tarea se escribe de forma que solo puede configurar los LEDs una vez que haya podido tomar su semáforo. Además entonces, habilita el siguiente semáforo.

Para que se pueda comenzar el ciclo debería tenerse algún semáforo consumible, de otra forma todas las tareas quedan en deadlock. Nos encargamos de esto en main(), una vez configurados los semáforos

```
xSem1 = xSemaphoreCreateBinary();
xSem2 = xSemaphoreCreateBinary();
xSem3 = xSemaphoreCreateBinary();
xSemaphoreGive(xSem1);

xTaskCreate(vTareaLED1, "LED1", 128, NULL, 1, NULL);
xTaskCreate(vTareaLED2, "LED2", 128, NULL, 1, NULL);
xTaskCreate(vTareaLED3, "LED3", 128, NULL, 1, NULL);

vTaskStartScheduler();
// ...
```

Notar que los semaforos se declararon globalmente

```
/* USER CODE BEGIN PV */
SemaphoreHandle_t xSem1;
SemaphoreHandle_t xSem2;
SemaphoreHandle_t xSem3;
/* USER CODE END PV */
```

7.2 Conclusiones

Para editar el orden en que sucede la secuencia, basta con hacer intercambios de semáforos en el código de las tareas. Por ejemplo si queremos LED1→LED3→LED3→...

```
void vTareaLED2(void *pvParameters){
    while (1){
        xSemaphoreTake( xSem3, portMAX_DELAY );
        HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LD4_GPIO_Port, LD4_Pin, GPIO_PIN_SET);
        xSemaphoreGive( xSem1 );
    }
}
void vTareaLED3(void *pvParameters){
    while (1){
        xSemaphoreTake( xSem2, portMAX_DELAY );
        HAL_GPIO_WritePin(LD4_GPIO_Port, LD4_Pin, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_SET);
        xSemaphoreGive( xSem3 );
    }
}
```

Se presenta una implementación con una sola tarea:

Primero es necesario pasarle ciertos parametros a esta única tarea para que sea creada con distintas configuraciones.

```
struct parametros_t {
    uint16_t Pin_prender;
    uint16_t Pin_apagar;
    SemaphoreHandle_t sem_to_take;
    SemaphoreHandle_t sem_to_give;
};
```

La tarea implementada es:

```
void vTareaLED(void *pvParameters){
    struct parametros_t parametros;
    parametros = *(struct parametros_t*) pvParameters;
    while (1){
        xSemaphoreTake( parametros.sem_to_take , portMAX_DELAY );

        HAL_GPIO_WritePin(GPIOD, parametros.Pin_prender , GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOD, parametros.Pin_apagar , GPIO_PIN_RESET);

        xSemaphoreGive( parametros.sem_to_give );
    }
}
```

Y en la main se configura

```
// ...
p1.Pin_prender = LD5_Pin;
p1.Pin_apagar  = LD3_Pin;
p1.sem_to_take = xSem1;
p1.sem_to_give = xSem2;

p2.Pin_prender = LD3_Pin;
p2.Pin_apagar  = LD4_Pin;
p2.sem_to_take = xSem2;
p2.sem_to_give = xSem3;

p3.Pin_prender = LD4_Pin;
p3.Pin_apagar  = LD5_Pin;
p3.sem_to_take = xSem3;
p3.sem_to_give = xSem1;

xTaskCreate(vTareaLED, "LED1", 128, &p1, 1, NULL);
xTaskCreate(vTareaLED, "LED2", 128, &p2, 1, NULL);
xTaskCreate(vTareaLED, "LED3", 128, &p3, 1, NULL);

vTaskStartScheduler();
```

8 Ejercicio 8

8.1 Implementación

Consideramos que el recurso con acceso exclusivo es todos los LEDs. Así que para iniciar cada secuencia es necesario tener el mutex.

```
void vTareaA(void *pvParameters){
    while(1){
        xSemaphoreTake(xLEDMutex, portMAX_DELAY);
        HAL_GPIO_WritePin(GPIOD, LD3_Pin, GPIO_PIN_SET);
        vTaskDelay(pdMS_TO_TICKS(100));
        HAL_GPIO_WritePin(GPIOD, LD3_Pin, GPIO_PIN_RESET);

        HAL_GPIO_WritePin(GPIOD, LD4_Pin, GPIO_PIN_SET);
        vTaskDelay(pdMS_TO_TICKS(100));
        HAL_GPIO_WritePin(GPIOD, LD4_Pin, GPIO_PIN_RESET);

        HAL_GPIO_WritePin(GPIOD, LD5_Pin, GPIO_PIN_SET);
        vTaskDelay(pdMS_TO_TICKS(100));
        HAL_GPIO_WritePin(GPIOD, LD5_Pin, GPIO_PIN_RESET);

        HAL_GPIO_WritePin(GPIOD, LD6_Pin, GPIO_PIN_SET);
        vTaskDelay(pdMS_TO_TICKS(100));
        HAL_GPIO_WritePin(GPIOD, LD6_Pin, GPIO_PIN_RESET);
        xSemaphoreGive(xLEDMutex);
    }
}

void vTareaB(void *pvParameters){
    while(1){
        xSemaphoreTake(xLEDMutex, portMAX_DELAY);
        for(int i = 0; i < 4; i++){
            HAL_GPIO_WritePin(GPIOD, LD3_Pin | LD4_Pin | LD5_Pin |
LD6_Pin, GPIO_PIN_SET);
            vTaskDelay(pdMS_TO_TICKS(150));

            HAL_GPIO_WritePin(GPIOD, LD3_Pin | LD4_Pin | LD5_Pin |
LD6_Pin, GPIO_PIN_RESET);
            vTaskDelay(pdMS_TO_TICKS(150));
        }
        xSemaphoreGive(xLEDMutex);
    }
}
```

En la main creamos las tareas como

```
/* USER CODE BEGIN WHILE */
xLEDMutex = xSemaphoreCreateMutex();

xTaskCreate(vTareaA, "LED1", 128, NULL, 1, NULL);
xTaskCreate(vTareaB, "LED2", 128, NULL, 1, NULL);

vTaskStartScheduler();
```

8.2 Conclusiones

En este ejercicio particular, el uso de un mutex garantiza la exclusión mutua sobre los LEDs, por lo que el efecto visual no se ve afectado por la concurrencia.

Pero al correr este programa en una placa, sucede que solo uno de los patrones es visible. El arbitraje esta funcionando, pero a pesar de esto la tarea que tomo el mutex primero, a pesar de que lo entrega, es capaz de tomarlo nuevamente antes que la otra tarea. Por lo que termina monopolizando el CPU.

Esto se puede cambiar agregando una espera activa al final que haga perder el resto del tick y el scheduler decida entregar el CPU a la tarea que todavia no tuvo CPU. Resultando que las tareas alternan una y una.

Otra opción para introducir competencia real, es introducir una cantidad aleatoria de ticks (en el rango 1,3) de espera previa a la instruccion de intentar tomar el s maforo.

Otra opci n

9 Ejercicio 9

9.1 Implementaci n

La tarea B se escribe de formas diferentes, seg n se maneje el bot n por polling o por interrupci n. La tarea A ser :

```
void vTareaA(void *pvParameters){
    while (1){
        HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_SET);
        HAL_Delay(50);
        HAL_GPIO_WritePin(LD4_GPIO_Port, LD4_Pin, GPIO_PIN_SET);
        HAL_Delay(50);
        HAL_GPIO_WritePin(LD5_GPIO_Port, LD5_Pin, GPIO_PIN_SET);
        HAL_Delay(50);
        HAL_GPIO_WritePin(LD6_GPIO_Port, LD6_Pin, GPIO_PIN_SET);
        HAL_Delay(50);
        HAL_GPIO_WritePin(LD3_GPIO_Port, LD3_Pin, GPIO_PIN_RESET);
        HAL_Delay(350);
        HAL_GPIO_WritePin(LD3_GPIO_Port, LD4_Pin, GPIO_PIN_RESET);
        HAL_Delay(350);
        HAL_GPIO_WritePin(LD3_GPIO_Port, LD5_Pin, GPIO_PIN_RESET);
```

```

        HAL_Delay(250);
        HAL_GPIO_WritePin(LD3_GPIO_Port, LD6_Pin, GPIO_PIN_RESET);
        HAL_Delay(250);
    }
}

```

Que simplemente controla un patron de luces, intencionalmente se usan esperas no bloqueantes para mostrar un problema del polling. Veamos la tarea B según se usa polling o interrupción

Manejo por interrupción

```

void vTareaB(void *pvParameters){
    TaskHandle_t handleTareaA = (TaskHandle_t) pvParameters;
    while (1){
        xSemaphoreTake(xButtonSemaphore, portMAX_DELAY);
        if ( AIsSuspended == pdTRUE ){
            vTaskResume(handleTareaA);
            AIsSuspended = pdFALSE;
        }
        else if ( AIsSuspended == pdFALSE ){
            vTaskSuspend(handleTareaA);
            AIsSuspended = pdTRUE;
        }
    }
}

```

Recibe el handle de la TareaA por parametro. En el mainloop toma el semáforo y solo consigue avanzar cuando la interrupción lo haga consumible. Y decide en que caso suspende o resume la tarea según la variable global AIsSuspended.

El callback de la interrupción es:

```

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    static TickType_t lastTick = 0;
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if (GPIO_Pin == GPIO_PIN_0){
        TickType_t now = xTaskGetTickCountFromISR();

        if ((now - lastTick) > pdMS_TO_TICKS(200)){
            xSemaphoreGiveFromISR(xButtonSemaphore, &
xHigherPriorityTaskWoken);
            lastTick = now;
        }

        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}

```

Siempre que se presiona el botón hace consumible el semáforo, salvo que haya controlado la interrupción recientemente (Debounce).

La main debe crear, las tareas con al menos la misma prioridad (Pero lo lógico es que B tenga mas prioridad). Porque a pesar de haberse dado el semáforo, tarea B podría no ser scheduleada si tareaA la supera en prioridad.

```

xButtonSemaphore = xSemaphoreCreateBinary();

xTaskCreate(vTareaA, "Boton", 128, NULL, 1, &handleTareaA);
xTaskCreate(vTareaB, "Boton", 128, (void*)handleTareaA, 1, NULL);

```

Manejo con polling Solo cambiaremos la tarea B y ya no necesitamos la función de callback.

```

void vTareaBpoller(void *pvParameters){
    TaskHandle_t handleTareaA = (TaskHandle_t) pvParameters;
    while (1){
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) {
            if ( AIsSuspended == pdTRUE ){
                vTaskResume(handleTareaA);
                AIsSuspended = pdFALSE;
            }
            else if ( AIsSuspended == pdFALSE ){
                vTaskSuspend(handleTareaA);
                AIsSuspended = pdTRUE;
            }
            vTaskDelay(pdMS_TO_TICKS(10));
        }
    }
}

```

Pero en este caso es necesario crear las tareas con la misma prioridad, si no, alguna monopoliza el CPU, por lo que: o no vemos el patrón de luces correctamente; o no se va a preguntar nunca por el estado del botón

9.2 Conclusiones

Tengamos en cuenta que usamos esperas no-bloqueantes en TareaA. Esto en el caso de polling provoca que sean frecuentes las ventanas de tiempo donde presionar el botón no tiene efecto alguno pues depende de que al momento de presionarse el botón, la TareaB sea la que se ejecuta. Por otro lado tocar el botón manejando con interrupción, provoca que siempre se vea la suspensión o reanudamiento de TareaA al presionar. La solución con mejor tiempo de respuesta es el polling si la TareaA utilizara esperas no-bloqueantes, dado que la interrupción implica realizar cambios de contexto para poder suspender o reanudar. Pero es claramente menos fiable hacer polling que hacer interrupción (presionar y no ver cambios).

10 Ejercicio 10

10.1 Implementación

Generamos un patrón muy rápido de leds con:

```
void vTareaA(void *pvParameters){
    TickType_t start = xTaskGetTickCount();
    const TickType_t duracion = pdMS_TO_TICKS(10000);

    while ((xTaskGetTickCount() - start) < duracion){
        HAL_GPIO_TogglePin(GPIOD, LD3_Pin);
        HAL_Delay(50);
        HAL_GPIO_TogglePin(GPIOD, LD6_Pin);
        HAL_Delay(50);
        HAL_GPIO_TogglePin(GPIOD, LD4_Pin);
        HAL_Delay(50);
        HAL_GPIO_TogglePin(GPIOD, LD5_Pin);
        HAL_Delay(50);
    }

    // Estado final: solo LD4 prendido
    HAL_GPIO_WritePin(GPIOD, LD4_Pin, GPIO_PIN_SET);

    vTaskDelete(NULL);
}
```

Notar que al final el LED_4 siempre se prende. Para exteriorizar el hecho de que la tarea se elimina, la crearemos junto a una tarea de menor prioridad que haga otro patrón mas lento.

```
void vTareaB(void *pvParameters){
    const TickType_t periodo = pdMS_TO_TICKS(800); // lento

    while (1){
        HAL_GPIO_WritePin(GPIOD, LD6_Pin, GPIO_PIN_SET);
        vTaskDelay(periodo);

        HAL_GPIO_WritePin(GPIOD, LD6_Pin, GPIO_PIN_RESET);
        vTaskDelay(periodo);
    }
}
```

Al crearlas con:

```
xTaskCreate(vTareaA, "Boton", 128, NULL, 2, NULL);
xTaskCreate(vTareaB, "Boton", 128, NULL, 1, NULL);

vTaskStartScheduler();
```

10.2 Conclusiones

Notar que mientras la tarea A exista, B nunca debería ejecutarse. Por lo que nunca vemos el patron de parpadeo lento, hasta que efectivamente pasaron los 10 segundos y A se elimina a si misma.